

RESUMO DO MÓDULO

Rotas no Express

As rotas são uma parte fundamental do desenvolvimento web usando o Express.js. Elas permitem que os desenvolvedores gerenciem diferentes URLs e solicitações HTTP dentro de um site ou sistema.

Vamos começar analisando um exemplo usando um servidor HTTP simples com Node.js:

javascript

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.setHeader('Content-Type', 'text/plain');

  if(req.url === '/'){
    res.statusCode = 200;
    res.end('Home page');
  } else if(req.url === '/about') {
    res.statusCode = 200;
    res.end('About page');
  } else {
    res.statusCode = 404;
    res.end('Page not found');
  }
});
```

```
server.listen(3000, () => {  
  console.log('Servidor em execução em http://localhost:3000/');  
});
```

Nesse exemplo, o servidor é configurado para responder a três rotas:

- A rota raiz (/) retorna o texto "Home page".
- A rota /about retorna o texto "About page".
- Qualquer outra rota não definida retorna o texto "Page not found".

Embora esse exemplo funcione, é necessário adicionar várias condições para gerenciar cada rota, o que torna o código difícil de manter conforme o número de rotas aumenta.

Agora, veja como o Express simplifica esse processo:

1. Primeiro, instale o Express no seu projeto:

```
npm install express --save
```

2. Em seguida, crie um novo script usando o Express para configurar as mesmas rotas:

javascript

```
const express = require('express');  
  
const app = express();  
  
app.get('/', (req, res) => {  
  res.send('Home page');
```

```
});
```

```
app.get('/about', (req, res) => {  
  res.send('About page');  
});
```

```
app.use((req, res) => {  
  res.status(404).send('Page not found');  
});
```

```
app.listen(3000, () => {  
  console.log('Servidor em execução em http://localhost:3000/');  
});
```

Neste exemplo, cada rota é criada usando métodos específicos do Express (`app.get()` e `app.use()`). Isso torna o código mais legível e fácil de manter.

Como o Express simplifica o gerenciamento de rotas?

No Express, o objeto `app` é usado para definir rotas. A primeira rota que definimos é a raiz (`/`), usando o método `get()`. A assinatura é a seguinte:

javascript

```
app.get('/', (req, res) => {  
  res.send('Home page');  
});
```

- O primeiro argumento é a definição da rota (/).
- O segundo argumento é uma função de callback com dois parâmetros:
 - req (request): representa os dados enviados pelo cliente.
 - res (response): representa a resposta que o serviço vai devolver.

Rota para páginas não encontradas

Para lidar com URLs que não correspondem a nenhuma rota definida, usamos um middleware com o método `use()`:

javascript

```
app.use((req, res) => {  
  res.status(404).send('Page not found');  
});
```

Quando usamos `app.use()`, o middleware será executado para todas as solicitações que não correspondem a nenhuma das rotas anteriores. Isso permite definir uma resposta padrão para "Página não encontrada".

Criando rotas com verbos HTTP diferentes

O Express também permite criar rotas usando outros verbos HTTP, como POST, PUT e DELETE. Cada um deles possui uma função específica:

javascript

```
app.post('/usuario', (req, res) => {  
  res.send('Usuário criado.');
```

```
});  
  
app.put('/usuario/:id', (req, res) => {  
  res.send(`Usuário ${req.params.id} atualizado.`);  
});
```

```
app.delete('/usuario/:id', (req, res) => {  
  res.send(`Usuário ${req.params.id} deletado.`);  
});
```

Essas rotas permitem realizar operações como criação (POST), atualização (PUT) e exclusão (DELETE) de dados, tornando o desenvolvimento de APIs mais intuitivo.
